

MediTrack DevOps Project — Terminal Commands Reference

Complete list of every command used in the project with explanations

SYSTEM & NAVIGATION COMMANDS

Command	Explanation
<code>brew --version</code>	Check if Homebrew (Mac package manager) is installed
<code>brew install hashicorp/tap/terraform</code>	Install Terraform from HashiCorp's official source
<code>brew install kubectl</code>	Install kubectl (Kubernetes command line tool)
<code>brew install helm</code>	Install Helm (Kubernetes package manager)
<code>brew install azure-cli</code>	Install Azure CLI (command line tool for Azure)
<code>cd ~</code>	Go to home directory (/Users/yourname)
<code>cd ~/Desktop</code>	Go to Desktop folder
<code>cd ~/Desktop/meditrack-devops</code>	Go to project folder
<code>cd ~/Desktop/meditrack-devops/terraform</code>	Go to terraform folder
<code>cd ~/Desktop/meditrack-devops/services/backend</code>	Go to backend service folder
<code>cd ~/Desktop/meditrack-devops/services/frontend</code>	Go to frontend service folder
<code>cd ~/Desktop/meditrack-devops/services/notification</code>	Go to notification service folder
<code>cd ~/Desktop/meditrack-devops/kubernetes/backend</code>	Go to backend kubernetes folder

Command	Explanation
<code>cd ~/Desktop/meditrack-devops/kubernetes/frontend</code>	Go to frontend kubernetes folder
<code>cd ~/Desktop/meditrack-devops/kubernetes/notification</code>	Go to notification kubernetes folder
<code>pwd</code>	Print Working Directory — shows your current location
<code>ls</code>	List files in current folder
<code>ls services/</code>	List files inside the services folder
<code>ls ~/Desktop/meditrack-devops/services/notification/</code>	List files in notification folder
<code>mkdir meditrack-devops</code>	Create a folder called meditrack-devops
<code>mkdir terraform</code>	Create a folder called terraform
<code>mkdir -p services/frontend services/backend services/notification</code>	Create multiple folders at once (-p creates parent folders too)
<code>mkdir -p kubernetes/backend kubernetes/frontend kubernetes/notification</code>	Create kubernetes folder structure
<code>code .</code>	Open current folder in VS Code
<code>uname -m</code>	Check your Mac chip type (arm64 = Apple Silicon, x86_64 = Intel)

AZURE CLI COMMANDS

Command	Explanation
<code>az login</code>	Login to Azure — opens browser for authentication
<code>az account show</code>	Show currently connected Azure account details

Command	Explanation
<code>az acr login --name mediatrackacr</code>	Login to Azure Container Registry so Docker can push/pull images
<code>az aks get-credentials --resource-group meditrack-rg --name meditrack-aks</code>	Download AKS cluster credentials to your Mac so kubectl can connect
<code>az aks get-credentials --resource-group meditrack-rg --name meditrack-aks --overwrite-existing</code>	Same as above but overwrites existing credentials
<code>az aks update --name meditrack-aks --resource-group meditrack-rg --attach-acr mediatrackacr</code>	Give AKS cluster permission to pull images from ACR
<code>az aks check-acr --name meditrack-aks --resource-group meditrack-rg --acr mediatrackacr</code>	Verify AKS can access ACR

TERRAFORM COMMANDS

Command	Explanation
<code>terraform --version</code>	Check Terraform version
<code>terraform init</code>	Initialize Terraform — downloads Azure plugin (run this first always)
<code>terraform plan</code>	Preview what Terraform will create/change/destroy before doing it
<code>terraform apply</code>	Actually create the infrastructure on Azure (asks for yes confirmation)
<code>terraform destroy</code>	Delete ALL infrastructure created by Terraform (asks for yes confirmation)

Terraform Command Order (Always Follow This):

```
terraform init → terraform plan → terraform apply
```

DOCKER COMMANDS

Command	Explanation
<code>docker --version</code>	Check Docker version
<code>docker images</code>	List all Docker images on your Mac
<code>docker build -t meditrack-backend:v1 .</code>	Build Docker image named meditrack-backend version v1 using Dockerfile in current folder
<code>docker build -t meditrack-frontend:v1 .</code>	Build frontend Docker image
<code>docker build -t meditrack-notification:v1 .</code>	Build notification Docker image
<code>docker run -p 5001:5000 meditrack-backend:v1</code>	Run backend container, map Mac port 5001 to container port 5000
<code>docker tag meditrack-backend:v1 mediatrackacr.azurecr.io/meditrack-backend:v1</code>	Tag backend image with ACR address for pushing
<code>docker tag meditrack-frontend:v1 mediatrackacr.azurecr.io/meditrack-frontend:v1</code>	Tag frontend image with ACR address
<code>docker tag meditrack-notification:v1 mediatrackacr.azurecr.io/meditrack-notification:v1</code>	Tag notification image with ACR address
<code>docker push mediatrackacr.azurecr.io/meditrack-backend:v1</code>	Push backend image to Azure Container Registry
<code>docker push mediatrackacr.azurecr.io/meditrack-frontend:v1</code>	Push frontend image to ACR
<code>docker push mediatrackacr.azurecr.io/meditrack-notification:v1</code>	Push notification image to ACR
<code>open /Applications/Docker.app</code>	Open Docker Desktop from Terminal

DOCKER BUILDX COMMANDS (Rebuild For Linux AMD64)

These are used to build images compatible with AKS (Linux AMD64) from an M3 Mac (ARM64)

Command	Explanation
<code>docker buildx build --platform linux/amd64 -t mediatrackacr.azurecr.io/meditrack-backend:v1 --push .</code>	Build backend image for Linux AMD64 AND push to ACR in one command
<code>docker buildx build --platform linux/amd64 -t mediatrackacr.azurecr.io/meditrack-frontend:v1 --push .</code>	Build frontend image for Linux AMD64 AND push to ACR
<code>docker buildx build --platform linux/amd64 -t mediatrackacr.azurecr.io/meditrack-notification:v1 --push .</code>	Build notification image for Linux AMD64 AND push to ACR

Why `--platform linux/amd64`?

M3 Mac builds images for	→ ARM64 (Apple Silicon)
AKS cluster runs on	→ Linux AMD64 (Intel/AMD)
<code>--platform linux/amd64</code>	→ forces correct architecture

KUBECTL COMMANDS (Kubernetes)

Command	Explanation
<code>kubectl version --client</code>	Check kubectl version
<code>kubectl get nodes</code>	List all nodes (servers) in the Kubernetes cluster
<code>kubectl get pods</code>	List all running pods (containers) in the cluster
<code>kubectl get pods -w</code>	Watch pods in real time (auto refreshes, press CTRL+C to stop)

Command	Explanation
<code>kubectl get services</code>	List all services and their external IPs
<code>kubectl apply -f ~/Desktop/meditrack-devops/kubernetes/backend/</code>	Deploy backend to Kubernetes using yaml files
<code>kubectl apply -f ~/Desktop/meditrack-devops/kubernetes/frontend/</code>	Deploy frontend to Kubernetes
<code>kubectl apply -f ~/Desktop/meditrack-devops/kubernetes/notification/</code>	Deploy notification to Kubernetes
<code>kubectl describe pod <pod-name></code>	Show detailed info about a pod including error events
<code>kubectl logs <pod-name></code>	Show logs from a pod — useful for debugging crashes
<code>kubectl rollout restart deployment meditrack-notification</code>	Restart a deployment to pick up new image

HELM COMMANDS (Coming Up Next)

Command	Explanation
<code>helm version</code>	Check Helm version
<code>helm create meditrack</code>	Create a new Helm chart structure
<code>helm install meditrack ./meditrack</code>	Install Helm chart to Kubernetes
<code>helm upgrade meditrack ./meditrack</code>	Upgrade existing Helm release
<code>helm list</code>	List all installed Helm releases
<code>helm uninstall meditrack</code>	Remove Helm release from Kubernetes

GIT COMMANDS (Coming Up — GitHub)

Command	Explanation
<code>git --version</code>	Check Git version
<code>git init</code>	Initialize a Git repository in current folder
<code>git add .</code>	Stage all files for commit
<code>git commit -m "message"</code>	Save staged files with a message
<code>git remote add origin <url></code>	Connect local repo to GitHub
<code>git push origin main</code>	Push code to GitHub
<code>git status</code>	Check what files have changed
<code>git log</code>	See commit history



COMPLETE PROJECT REBUILD COMMANDS

Run these every time you start a new session (after terraform destroy the night before)

```
bash
```

```
# Step 1 - Navigate to terraform and rebuild infrastructure
cd ~/Desktop/meditrack-devops/terraform
terraform apply

# Step 2 - Login to ACR
az acr login --name mediatrackacr

# Step 3 - Rebuild backend image for Linux AMD64
cd ~/Desktop/meditrack-devops/services/backend
docker buildx build --platform linux/amd64 -t mediatrackacr.azurecr.io/meditrack-bac

# Step 4 - Rebuild frontend image
cd ~/Desktop/meditrack-devops/services/frontend
docker buildx build --platform linux/amd64 -t mediatrackacr.azurecr.io/meditrack-fro

# Step 5 - Rebuild notification image
cd ~/Desktop/meditrack-devops/services/notification
docker buildx build --platform linux/amd64 -t mediatrackacr.azurecr.io/meditrack-not

# Step 6 - Attach ACR to AKS
az aks update --name meditrack-aks --resource-group meditrack-rg --attach-acr mediat

# Step 7 - Get cluster credentials
az aks get-credentials --resource-group meditrack-rg --name meditrack-aks --overwrit

# Step 8 - Deploy all services
kubectl apply -f ~/Desktop/meditrack-devops/kubernetes/backend/
kubectl apply -f ~/Desktop/meditrack-devops/kubernetes/frontend/
kubectl apply -f ~/Desktop/meditrack-devops/kubernetes/notification/

# Step 9 - Verify everything running
kubectl get pods
kubectl get services
```

END OF DAY COMMANDS

Always run these before closing your laptop

```
bash
```



```
# Step 1 - Destroy Azure infrastructure (saves credits!)
cd ~/Desktop/meditrack-devops/terraform
terraform destroy

# Step 2 - Quit Docker Desktop
# Click whale icon in menu bar → Quit Docker Desktop
```

QUICK STATUS CHECK COMMANDS

Run these anytime to see what's happening

```
bash

kubectl get pods           # Are all pods running?
kubectl get services       # What's the frontend public IP?
kubectl get nodes          # Is the cluster node healthy?
docker images              # What images are on my Mac?
az account show            # Am I logged into Azure?
terraform --version        # Is Terraform installed?
```

COMMON ERRORS AND FIXES

Error	Cause	Fix
<code>command not found: docker</code>	Docker Desktop not running	Open Docker Desktop, wait for whale
<code>command not found: terraform</code>	Terraform not installed	<code>brew install hashicorp/tap/terraform</code>
<code>ErrImagePull</code>	Wrong image architecture or ACR permissions	Rebuild with <code>--platform linux/amd64</code>
<code>CrashLoopBackOff</code>	App crashing inside container	Run <code>kubectl logs <pod-name></code> to see why
<code>address already in use</code>	Port occupied by another app	Use different port e.g. <code>-p 5001:5000</code>

Error	Cause	Fix
<code>No such file or directory</code>	Missing app.py in service folder	Create the missing file in correct folder
<code>401 Unauthorized</code>	ACR login expired	Run <code>az acr login --name mediatrackacr</code>
<code>chdir: no such file or directory</code>	Wrong path with -chdir	Use <code>cd</code> first then run command

Generated during MediTrack DevOps Project — Azure, Docker, Kubernetes, Terraform